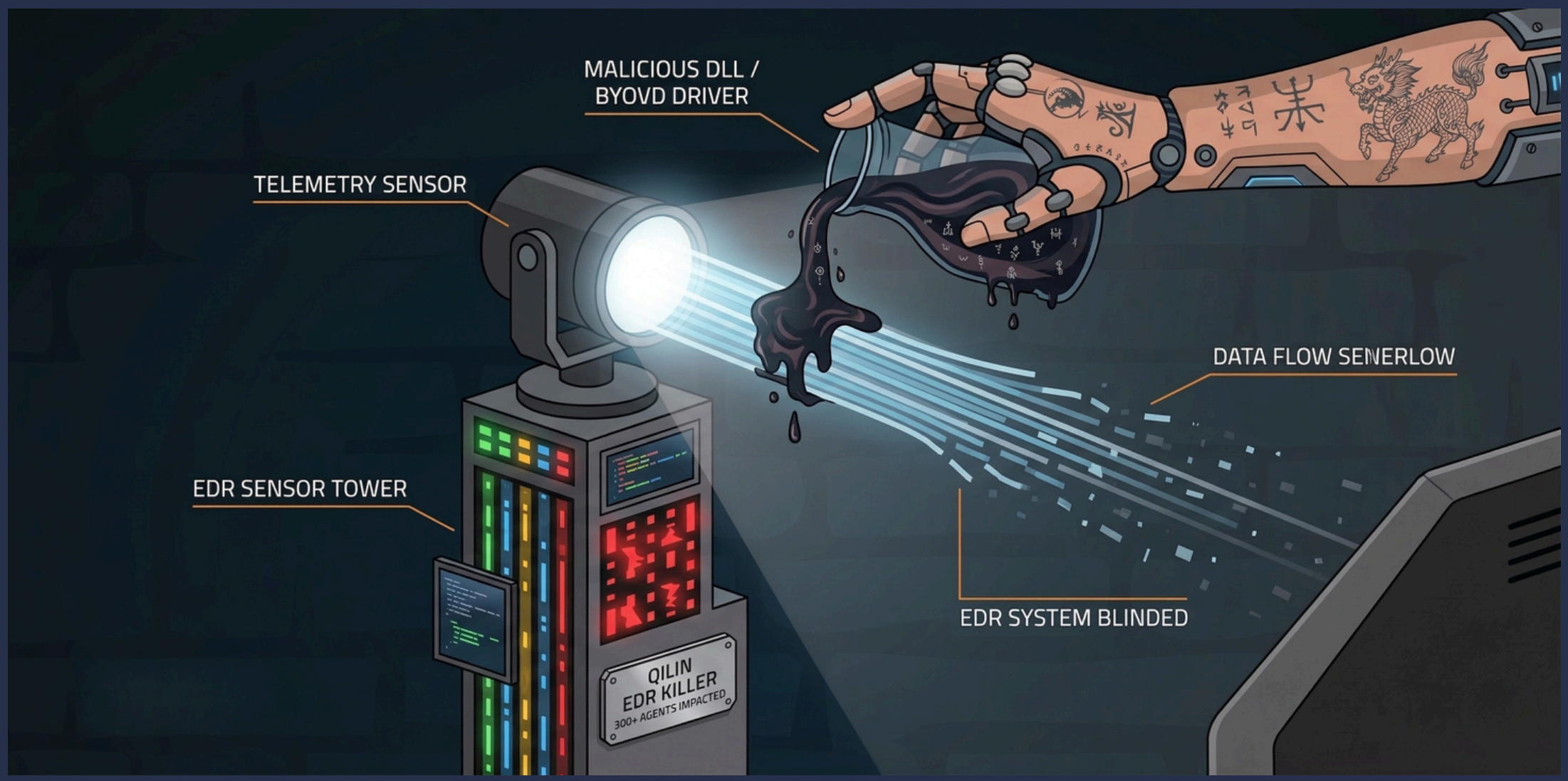
Qilin's Infection Chain: How a Ransomware Dismantles 300+ EDR Solutions



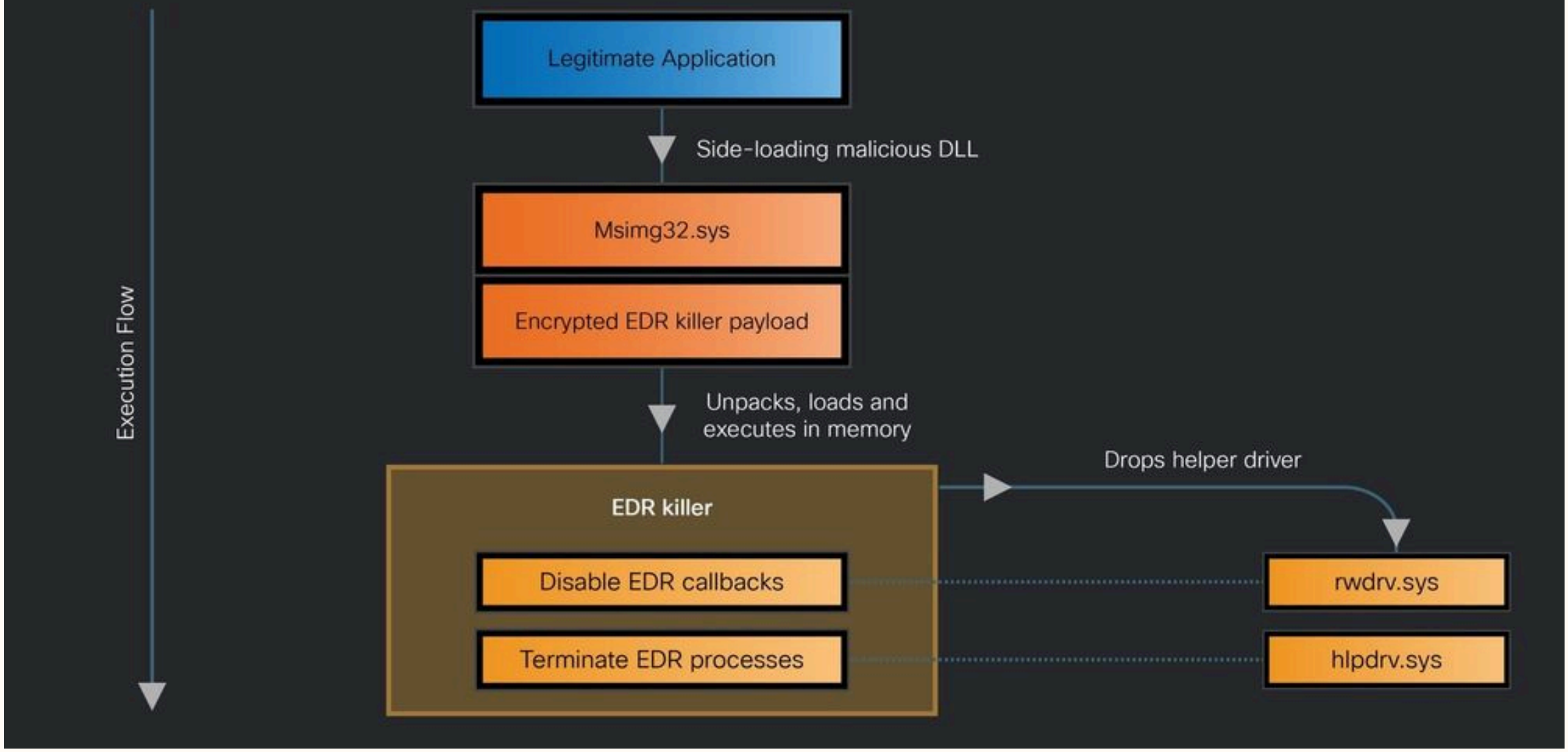
Memori CTI

Qilin's EDR Killer

As endpoint detection and response tools become the standard for enterprise security, threat actors are continuously evolving their initial access phases to neutralize these defenses. The Qilin ransomware operation has recently been observed deploying a highly sophisticated infection chain. This campaign utilizes a customized EDR killer capable of disabling over 300 security drivers across various vendors.

The attack architecture relies heavily on evasion, memory manipulation, and kernel-level exploitation to systematically dismantle security software before the final ransomware payload is delivered.

Infection chain overview



Initial Access and In-Memory Execution

The infection begins with a DLL sideloading technique. A legitimate executable, such as **FoxitPDFReader.exe**, is used to load a malicious version of **msimg32.dll**. To maintain the illusion of normal operation and avoid application crashes, the malicious DLL forwards standard API calls to the legitimate Windows library located in the System32 directory.
The actual malicious payload triggers directly from the DllMain function. A multi-stage loader then decrypts and executes the core EDR killer component entirely in memory. This memory-only execution model ensures that the decrypted payload never touches the physical disk, severely limiting the artifacts available for traditional signature-based detection.

```
.rdata:00000001801403D8 41 6C 70 68 61 42 6C 65 6E 64…aAlphablend     db 'AlphaBlend',0       ; DATA XREF: .rdata:off_18014039C↑o
.rdata:00000001801403E3                                 ; Exported entry   1. AlphaBlend
.rdata:00000001801403E3                                                 public AlphaBlend
.rdata:00000001801403E3 43 3A 5C 57 69 6E 64 6F 77 73 AlphaBlend      db 'C:\Windows\System32\msimg32.AlphaBlend',0
.rdata:00000001801403E3 5C 53 79 73 74 65 6D 33 32 5C…                                  ; DATA XREF: .rdata:off_180140388↑o
.rdata:000000018014040A 44 6C 6C 49 6E 69 74 69 61 6C…aDllinitialize  db 'DllInitialize',0    ; DATA XREF: .rdata:off_18014039C↑o
.rdata:0000000180140418                                 ; Exported entry   2. DllInitialize
.rdata:0000000180140418                                                 public DllInitialize
.rdata:0000000180140418 43 3A 5C 57 69 6E 64 6F 77 73 DllInitialize   db 'C:\Windows\System32\msimg32.DllInitialize',0
.rdata:0000000180140418 5C 53 79 73 74 65 6D 33 32 5C…                                  ; DATA XREF: .rdata:off_180140388↑o
.rdata:0000000180140442 47 72 61 64 69 65 6E 74 46 69…aGradientfill   db 'GradientFill',0     ; DATA XREF: .rdata:off_18014039C↑o
.rdata:000000018014044F                                 ; Exported entry   3. GradientFill
.rdata:000000018014044F                                                 public GradientFill
.rdata:000000018014044F 43 3A 5C 57 69 6E 64 6F 77 73 GradientFill    db 'C:\Windows\System32\msimg32.GradientFill',0
.rdata:000000018014044F 5C 53 79 73 74 65 6D 33 32 5C…                                  ; DATA XREF: .rdata:off_180140388↑o
.rdata:0000000180140478 54 72 61 6E 73 70 61 72 65 6E…aTransparentblt db 'TransparentBlt',0   ; DATA XREF: .rdata:off_18014039C↑o
.rdata:0000000180140487                                 ; Exported entry   4. TransparentBlt
.rdata:0000000180140487                                                 public TransparentBlt
.rdata:0000000180140487 43 3A 5C 57 69 6E 64 6F 77 73 TransparentBlt  db 'C:\Windows\System32\msimg32.TransparentBlt',0
.rdata:0000000180140487 5C 53 79 73 74 65 6D 33 32 5C…                                  ; DATA XREF: .rdata:off_180140388↑o
.rdata:00000001801404B2 76 53 65 74 44 64 72 61 77 66…aVsetddrawflag  db 'vSetDdrawflag',0    ; DATA XREF: .rdata:off_18014039C↑o
.rdata:00000001801404C0                                 ; Exported entry   5. vSetDdrawflag
.rdata:00000001801404C0                                                 public vSetDdrawflag
.rdata:00000001801404C0 43 3A 5C 57 69 6E 64 6F 77 73 vSetDdrawflag   db 'C:\Windows\System32\msimg32.vSetDdrawflag',0
.rdata:00000001801404C0 5C 53 79 73 74 65 6D 33 32 5C…                                  ; DATA XREF: .rdata:off_180140388↑o
.rdata:00000001801404EA 00 00 00 00 00 00 00 00 00 00                   align 1000h
.rdata:00000001801404EA 00 00 00 00 00 00 00 00 00 00 _rdata            ends
```

User-Mode Evasion and Syscall Bypassing

Before the malware attempts to interact with the kernel, the loader employs advanced user-mode evasion tactics to hide its activity from local sensors.

- **ETW Suppression:** Event Tracing for Windows is suppressed at runtime. This deprives EDR solutions of the telemetry required for behavioral analysis.
- **Halo's Gate Implementation (Syscall Bypassing):** Security products typically place hooks inside ntdll.dll to monitor critical API calls. To bypass these, the malware utilizes a Halo's Gate technique. By scanning the **ntdll.dll** file forward and backward, it locates unhooked syscall stubs. It then repurposes these clean stubs to transition directly to the kernel, circumventing user-mode monitoring entirely.

- **Exception Handling Obfuscation:** Control flow is obscured using Structured Exception Handling and Vectored Exception Handling mechanisms. The loader makes the **.mrdata** section of **ntdll.dll** writable and overwrites the exception dispatcher callback pointer, redirecting execution to its own custom routines.
- **Anti-Analysis Checks:** The malware actively monitors for hardware breakpoints on **KiUserExceptionDispatcher.** If a debugger is detected, the process is deliberately crashed to prevent security researchers from analyzing the execution flow.

```
do                                      // ========== Build Slot-Policy-table for SysCall stubs ==========
{
  NamePtr = (char *)ntDllBase + *(unsigned int *)&AdressOfNames[4 * --AddressOfNamesIndex];
  stub_idx = 'tN';
  if ( *(_WORD *)NamePtr != 'tN' )          // only functions starting with 'Nt' aka Syscal stubs
    goto set_idx_continue;
  if ( (unsigned int)hash_a1_2(NamePtr) == 0x5FC9AA50 )// NtTraceEvent
  {
    stub_idx = (unsigned __int64)(unsigned int)((_DWORD)ntDllBase// map = (NtTraceEventFuncVA - BaseOfCode) / 16;
                                          // maps each NT function to a 16-byte slot relative to the code start. var = Slot-Policy-table
                                          + *(_DWORD *)&AddressOfFunctions[4 * ordinals[AddressOfNamesIndex]]
                                          - LODWORD(a1_ctx_ff8->BaseOfCode)) >> 4;
    map[stub_idx] = 2;                    // If NtTraceEvent set policy to 2
next_round:
    AdressOfNames = AdressOfNamesOrg;
set_idx_continue:
    api_func_indx = 4;
    continue;
  }
  if ( (unsigned int)hash_a1_2(NamePtr) == 0x7C99D9A8 )// NtTraceControl
  {
    stub_idx = (unsigned __int64)(unsigned int)((_DWORD)ntDllBase// map = (NtTraceControlFuncVA - BaseOfCode) / 16;
                                          + *(_DWORD *)&AddressOfFunctions[4 * ordinals[AddressOfNamesIndex]]
                                          - LODWORD(a1_ctx_ff8->BaseOfCode)) >> 4;
    map[stub_idx] = 3;                    // If NtTraceControl set policy to 3
    goto next_round;
  }
  ordinalsIndex = ordinals[AddressOfNamesIndex];
  hash_ptr_org = hash_a1_2(NamePtr);
  AdressOfNames = AdressOfNamesOrg;
  hash_ptr1 = hash_ptr_org == 0xE7295650;     // NtAlpcSendWaitReceivePort
  FuncRVAaddr = *(_DWORD *)&AddressOfFunctions[4 * ordinalsIndex];
  api_func_indx = 4;
  stub_idx = (unsigned __int64)(unsigned int)((_DWORD)ntDllBase + FuncRVAaddr - LODWORD(a1_ctx_ff8->BaseOfCode)) >> 4;// map = (NtAlpcSendWaitReceivePortFuncVA - BaseOfCode) / 16;
  if ( hash_ptr1 )
    map[stub_idx] = 4;                    // If NtAlpcSendWaitReceivePort set policy to 4
  else
    map[stub_idx] = 1;                    // If Other Nt-stub set policy to 1
}
while ( AddressOfNamesIndex );            // Go through all exports of ntdll.dll
```

Escalation via Vulnerable Drivers

To achieve the goal of terminating protected EDR processes, the malware must escalate its privileges to the kernel level. It accomplishes this by dropping two specific helper drivers onto the system.

- **rwdrv.sys (The Vulnerable Driver):** This is a renamed version of **ThrottleStop.sys**, a legitimate driver signed by TechPowerUp LLC (commonly used in hardware monitoring tools like GPU-Z). Because it has a valid digital signature, it bypasses Windows Driver Signature Enforcement. However, this driver exposes highly privileged IOCTLs that allow arbitrary read/write access to physical memory. Qilin exploits this driver to directly manipulate kernel structures, executing a classic Bring Your Own Vulnerable Driver (BYOVD) attack.
- **hlpdrv.sys:** A dedicated driver used exclusively to terminate protected EDR processes via **IOCTL code 0x2222008**, bypassing standard Windows process protection mechanisms.

Neutralizing Kernel Monitoring Callbacks

Using the vulnerable driver's memory access, the malware targets over 300 EDR drivers by unregistering their critical monitoring callbacks, effectively blinding the security tools. To prevent system crashes (Blue Screen of Death) during this deep memory manipulation, it temporarily suspends Windows Code Integrity enforcement by overwriting the **CiValidateImageHeader** callback, restoring it afterward to hide its tracks.

Defense Implications

The Qilin campaign illustrates a strategic shift in ransomware deployment. Neutralizing the defense stack is treated as a primary operational objective rather than an afterthought. Protecting against these advanced evasion and vulnerable driver techniques requires environments to enforce strict driver blocklists, such as the Microsoft WDAC blocklist, alongside robust monitoring for anomalous DLL sideloading behaviors.